

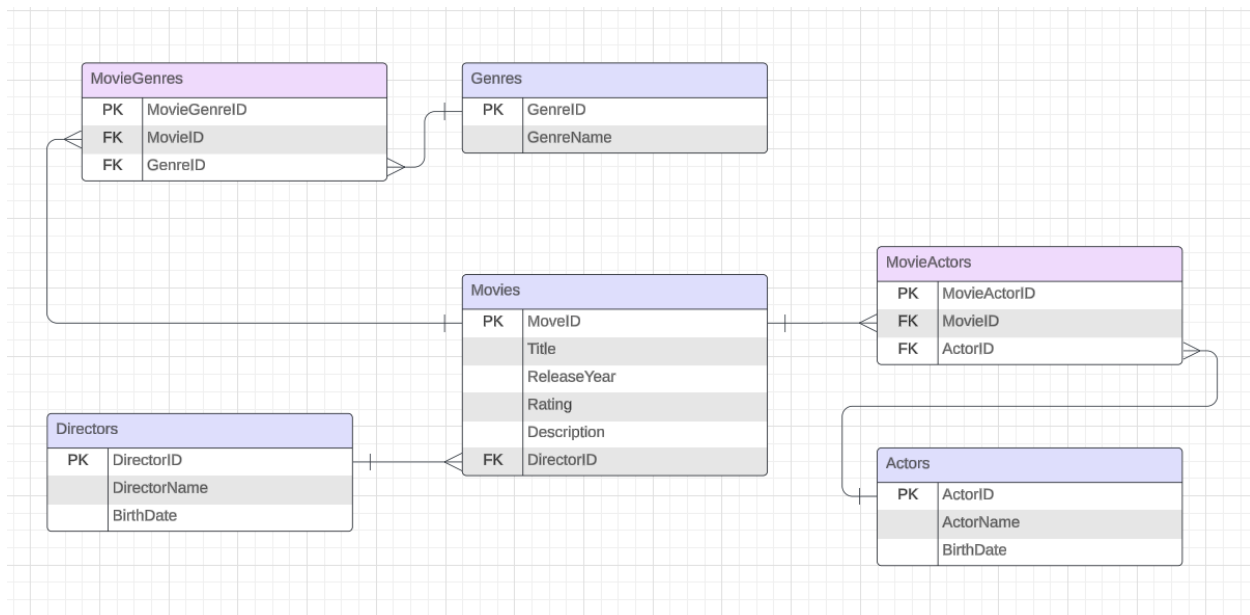
Project 1: Report**Section 1:****ER Model Diagram**

Figure 1 ER Diagram representing a database for movies. Purple indicates primary entities and Pink indicates junction tables. The junctions are created to mitigate many-to-many relationships.

Section 2:

Database Selection and Creation

The database *movies.db* was created using SQLite. An SQL script named *schema.sql* was written to Create the tables and overall structure of the database.

```
schema.sql
1  -- SQLite
2  CREATE TABLE Movies (
3      MovieID INTEGER PRIMARY KEY,           -- Unique identifier for each movie
4      Title TEXT NOT NULL,                   -- Title of the movie
5      ReleaseYear INTEGER,                   -- Year the movie was released
6      Rating REAL,                           -- Rating of the movie (e.g., 7.5)
7      Description TEXT,                       -- Brief description of the movie
8      DirectorID INTEGER,                    -- Foreign key referencing Directors
9      FOREIGN KEY (DirectorID) REFERENCES Directors(DirectorID)
10 );
11
12 CREATE TABLE Genres (
13     GenreID INTEGER PRIMARY KEY,            -- Unique identifier for each genre
14     GenreName TEXT NOT NULL                 -- Name of the genre (e.g., Action, Comedy)
15 );
16
17 CREATE TABLE Directors (
18     DirectorID INTEGER PRIMARY KEY,          -- Unique identifier for each director
19     DirectorName TEXT NOT NULL,              -- Name of the director
20     BirthDate TEXT                           -- Birthdate of the director in YYYY-MM-DD format
21 );
22
23 CREATE TABLE Actors (
24     ActorID INTEGER PRIMARY KEY,             -- Unique identifier for each actor
25     ActorName TEXT NOT NULL,                 -- Name of the actor
26     BirthDate TEXT                           -- Birthdate of the actor in YYYY-MM-DD format
27 );
28
29 CREATE TABLE MovieGenres (
30     MovieGenreID INTEGER PRIMARY KEY,        -- Unique identifier for each movie-genre pair
31     MovieID INTEGER NOT NULL,                -- Foreign key referencing Movies
32     GenreID INTEGER NOT NULL,                -- Foreign key referencing Genres
33     FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
34     FOREIGN KEY (GenreID) REFERENCES Genres(GenreID)
35 );
36
37 CREATE TABLE MovieActors (
38     MovieActorID INTEGER PRIMARY KEY,        -- Unique identifier for each movie-actor pair
39     MovieID INTEGER NOT NULL,                -- Foreign key referencing Movies
40     ActorID INTEGER NOT NULL,                -- Foreign key referencing Actors
41     FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
42     FOREIGN KEY (ActorID) REFERENCES Actors(ActorID)
43 );
44
```

Figure 2 Our schema.sql script is used to generate the tables within our database. Note the Primary Keys, Foreign Keys, and data types.

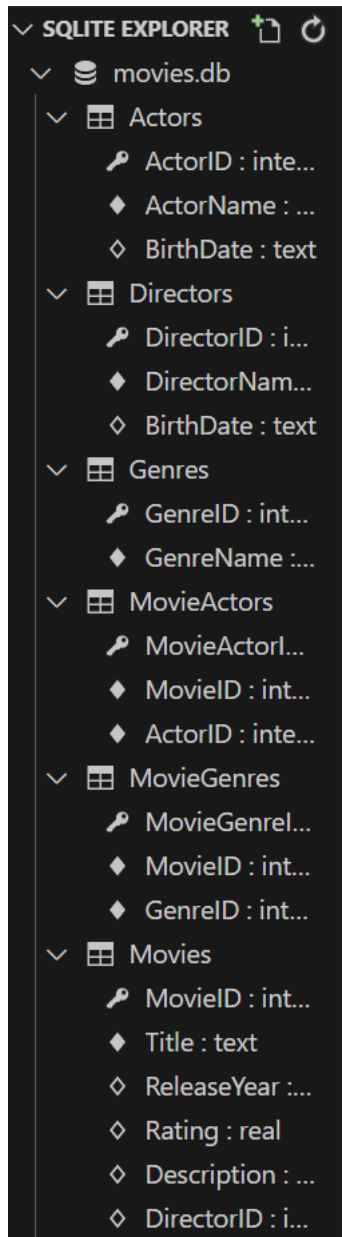


Figure 3 These are the tables generated for our database following the running of the queries within schema.sql

Section 3:

Data Population

We used python to populate our database. We named the script *populate_db.py*

```
1 import sqlite3
2
3 # Connect to the SQLite database
4 connection = sqlite3.connect('movies.db')
5 cursor = connection.cursor()
6
7 # Revised data to be inserted
8 movies = [
9     (1, "Jaws", 1975, 8.0, "A giant great white shark terrorizes a beach town.", 1),
10    (2, "E.T. the Extra-Terrestrial", 1982, 7.8, "A troubled child summons the courage to help a friendly alien escape Earth.", 1),
11    (3, "Goodfellas", 1990, 8.7, "The story of Henry Hill and his life in the mob.", 2),
12    (4, "Taxi Driver", 1976, 8.3, "A mentally unstable Vietnam War veteran becomes a vigilante.", 2),
13    (5, "The Godfather", 1972, 9.2, "The aging patriarch of an organized crime dynasty transfers control to his reluctant son.", 3),
14    (6, "Apocalypse Now", 1979, 8.4, "During the Vietnam War, Captain Willard is sent to assassinate a rogue colonel.", 3),
15    (7, "Pulp Fiction", 1994, 8.9, "The lives of two mob hitmen intertwine with stories of crime.", 4),
16    (8, "Kill Bill: Volume 1", 2003, 8.1, "A former assassin awakens from a coma to seek revenge on her ex-colleagues.", 4),
17    (9, "Inception", 2010, 8.8, "A thief who steals corporate secrets using dream-sharing technology.", 5),
18    (10, "Dunkirk", 2017, 7.9, "Allied soldiers are surrounded by enemy forces and evacuated during a fierce battle in World War II.", 5),
19    (11, "Gladiator", 2000, 8.5, "A former Roman General sets out to exact vengeance against the corrupt emperor.", 6),
20    (12, "Alien", 1979, 8.4, "The crew of a commercial spacecraft encounter a deadly alien after investigating an unknown transmission.", 6),
21    (13, "Fight Club", 1999, 8.8, "An insomniac office worker and a soap maker form an underground fight club.", 7),
22    (14, "Se7en", 1995, 8.6, "Two detectives hunt a serial killer who uses the seven deadly sins as his modus operandi.", 7), # Add a comma here
23    (15, "Spider-Man: No Way Home", 2021, 8.3, "Peter Parker faces new challenges and consequences of being Spider-Man.", 8),
24    (16, "Dune", 2021, 8.2, "A noble family becomes embroiled in a war for control over the galaxy's most valuable asset.", 9),
25    (17, "Unbreakable", 2000, 7.3, "A man discovers he has superhero-like abilities after surviving a train crash.", 10),
26    (18, "Once Upon a Time in Hollywood", 2019, 7.6, "An aging actor and his stunt double navigate Hollywood in the late 1960s.", 4),
27    (19, "Inglourious Basterds", 2009, 8.3, "A group of Jewish soldiers in Nazi-occupied France plan to assassinate Nazi leaders.", 4),
28    (20, "Jurassic Park", 1993, 8.1, "Scientists clone dinosaurs to create a theme park, which goes horribly wrong.", 1),
29    (21, "Blade Runner", 1982, 8.1, "In a dystopian future, synthetic humans are hunted down by special police operatives.", 3),
30    (22, "Schindler's List", 1993, 9.0, "In German-occupied Poland during World War II, Oskar Schindler gradually becomes concerned for his Jewish workforce after witnessing their persecution.", 3),
31    (23, "Reservoir Dogs", 1992, 8.3, "After a botched robbery, tensions escalate among criminals as they suspect a mole.", 4),
32    (24, "The Hateful Eight", 2015, 7.8, "In the aftermath of the American Civil War, eight strangers seek refuge during a blizzard.", 4)
33 ]
34
35 genres = [
36     (1, "Drama"),
37     (2, "Crime"),
38     (3, "Thriller"),
39     (4, "Adventure"),
40     (5, "Sci-Fi"),
41     (6, "Action"),
42     (7, "Horror"),
43     (8, "War"),
44     (9, "Superhero"),
45     (10, "Epic"),
46     (11, "Comedy")
47 ]
```

Figure 4 First we connect to the database and then set our cursor. Then we create movies and genres lists.

```

49 directors = [
50     (1, "Steven Spielberg", "1946-12-18"),
51     (2, "Martin Scorsese", "1942-11-17"),
52     (3, "Francis Ford Coppola", "1939-04-07"),
53     (4, "Quentin Tarantino", "1963-03-27"),
54     (5, "Christopher Nolan", "1970-07-30"),
55     (6, "Ridley Scott", "1937-11-30"),
56     (7, "David Fincher", "1962-08-28"),
57     (8, "Jon Watts", "1981-10-28"),
58     (9, "Denis Villeneuve", "1967-10-03"),
59     (10, "M. Night Shyamalan", "1970-08-06")
60 ]
61
62 actors = [
63     (1, "Robert Shaw", "1927-04-09"), # Starred in "Jaws"
64     (2, "Drew Barrymore", "1975-02-22"), # Starred in "E.T. the Extra-Terrestrial"
65     (3, "Ray Liotta", "1954-12-18"), # Starred in "Goodfellas"
66     (4, "Robert De Niro", "1943-08-17"), # Starred in "Taxi Driver" and "Goodfellas"
67     (5, "Marlon Brando", "1924-04-03"), # Starred in "The Godfather" and "Apocalypse Now"
68     (6, "Martin Sheen", "1940-04-03"), # Starred in "Apocalypse Now"
69     (7, "John Travolta", "1954-02-18"), # Starred in "Pulp Fiction"
70     (8, "Uma Thurman", "1970-04-29"), # Starred in "Kill Bill: Volume 1" and "Pulp Fiction"
71     (9, "Leonardo DiCaprio", "1974-11-11"), # Starred in "Inception"
72     (10, "Fionn Whitehead", "1997-01-18"), # Starred in "Dunkirk"
73     (11, "Russell Crowe", "1974-04-07"), # Starred in "Gladiator"
74     (12, "Sigourney Weaver", "1949-10-08"), # Starred in "Alien"
75     (13, "Edward Norton", "1969-08-18"), # Starred in "Fight Club"
76     (14, "Morgan Freeman", "1937-06-01"), # Starred in "Se7en"
77     (15, "Brad Pitt", "1963-12-18"), # Starred in "Fight Club" and "Se7en"
78     (16, "Samuel L. Jackson", "1948-12-21"), # Starred in "Pulp Fiction" and "Unbreakable"
79     (17, "Bruce Willis", "1955-03-19"), # Starred in "Pulp Fiction" and "Unbreakable"
80     (18, "Robert Duvall", "1931-01-05"), # Starred in "Apocalypse Now"
81     (19, "Tom Holland", "1996-06-01"), # Starred in "Spider-Man: No Way Home"
82     (20, "Timothée Chalamet", "1995-12-27"), # Starred in "Dune"
83     (21, "Harrison Ford", "1942-07-13"), # Starred in "Blade Runner"
84 ]
85

```

Figure 5 Here we create our directors and actors lists.

```

86 movie_genres = [
87     (1, 1, 7), # Jaws - Horror
88     (2, 1, 4), # Jaws - Adventure
89     (3, 2, 4), # E.T. the Extra-Terrestrial - Adventure
90     (4, 2, 5), # E.T. the Extra-Terrestrial - Sci-Fi
91     (5, 3, 2), # Goodfellas - Crime
92     (6, 3, 3), # Goodfellas - Drama
93     (7, 4, 3), # Taxi Driver - Thriller
94     (8, 4, 1), # Taxi Driver - Drama
95     (9, 5, 2), # The Godfather - Crime
96     (10, 5, 1), # The Godfather - Drama
97     (11, 6, 3), # Apocalypse Now - Thriller
98     (12, 6, 8), # Apocalypse Now - War
99     (13, 7, 2), # Pulp Fiction - Crime
100    (14, 7, 3), # Pulp Fiction - Thriller
101    (15, 7, 12), # Pulp Fiction - Comedy
102    (16, 8, 6), # Kill Bill: Volume 1 - Action
103    (17, 8, 3), # Kill Bill: Volume 1 - Thriller
104    (18, 9, 5), # Inception - Sci-Fi
105    (19, 9, 3), # Inception - Thriller
106    (20, 10, 8), # Dunkirk - War
107    (21, 10, 3), # Dunkirk - Thriller
108    (22, 11, 6), # Gladiator - Action
109    (23, 11, 1), # Gladiator - Drama
110    (24, 12, 7), # Alien - Horror
111    (25, 12, 5), # Alien - Sci-Fi
112    (26, 13, 3), # Fight Club - Thriller
113    (27, 13, 1), # Fight Club - Drama
114    (28, 14, 2), # Se7en - Crime
115    (29, 14, 3), # Se7en - Thriller
116    (30, 15, 6), # Spider-Man: No Way Home - Action
117    (31, 15, 4), # Spider-Man: No Way Home - Adventure
118    (32, 15, 10), # Spider-Man: No Way Home - Superhero
119    (33, 16, 5), # Dune - Sci-Fi
120    (34, 16, 4), # Dune - Adventure
121    (35, 16, 11), # Dune - Epic
122    (36, 17, 1), # Unbreakable - Drama
123    (37, 17, 10), # Unbreakable - Superhero
124    (38, 18, 1), # Once Upon a Time in Hollywood - Drama
125    (39, 18, 4), # Once Upon a Time in Hollywood - Adventure
126    (40, 18, 12), # Once Upon a Time in Hollywood - Comedy
127    (41, 19, 2), # Inglourious Basterds - War
128    (42, 19, 3), # Inglourious Basterds - Drama
129    (43, 19, 12), # Inglourious Basterds - Comedy
130    (44, 20, 4), # Jurassic Park - Adventure
131    (45, 20, 5), # Jurassic Park - Sci-Fi

```

Figure 6 Here we begin our movies_genres list that will be used for our junction table.

```

132     (46, 21, 5), # Blade Runner - Sci-Fi
133     (47, 21, 3), # Blade Runner - Thriller
134     (48, 23, 2), # Reservoir Dogs - Crime
135     (49, 23, 3) # Reservoir Dogs - Thriller
136 ]
137
138 movie_actors = [
139     (1, 1, 1), # "Jaws" with Robert Shaw
140     (2, 2, 2), # "E.T. the Extra-Terrestrial" with Drew Barrymore
141     (3, 3, 3), # "Goodfellas" with Ray Liotta
142     (4, 3, 4), # "Goodfellas" with Robert De Niro
143     (5, 4, 4), # "Taxi Driver" with Robert De Niro
144     (6, 5, 5), # "The Godfather" with Marlon Brando
145     (7, 5, 18), # "The Godfather" with Robert Duvall
146     (8, 6, 5), # "Apocalypse Now" with Marlon Brando
147     (9, 6, 6), # "Apocalypse Now" with Martin Sheen
148     (10, 6, 18), # "Apocalypse Now" with Robert Duvall
149     (11, 7, 7), # "Pulp Fiction" with John Travolta
150     (12, 7, 8), # "Pulp Fiction" with Uma Thurman
151     (13, 7, 16), # "Pulp Fiction" with Samuel L. Jackson
152     (14, 7, 17), # "Pulp Fiction" with Bruce Willis
153     (15, 8, 8), # "Kill Bill: Volume 1" with Uma Thurman
154     (16, 9, 9), # "Inception" with Leonardo DiCaprio
155     (17, 10, 10), # "Dunkirk" with Fionn Whitehead
156     (18, 11, 11), # "Gladiator" with Russell Crowe
157     (19, 12, 12), # "Alien" with Sigourney Weaver
158     (20, 13, 13), # "Fight Club" with Edward Norton
159     (21, 13, 15), # "Fight Club" with Brad Pitt
160     (22, 14, 15), # "Se7en" with Brad Pitt
161     (23, 14, 14), # "Se7en" with Morgan Freeman
162     (24, 15, 19), # "Spider-Man: No Way Home" with Tom Holland
163     (25, 16, 28), # "Dune" with Timothée Chalamet
164     (26, 17, 17), # "Unbreakable" with Bruce Willis
165     (27, 17, 16), # "Unbreakable" with Samuel L. Jackson
166     (28, 18, 9), # "Once Upon a Time in Hollywood" with Leonardo DiCaprio
167     (29, 18, 15), # "Once Upon a Time in Hollywood" with Brad Pitt
168     (30, 19, 15), # "Inglourious Basterds" with Brad Pitt
169     (31, 20, 16), # "Jurassic Park" with Samuel L. Jackson
170     (32, 21, 21), # "Blade Runner" with Harrison Ford
171     (33, 24, 16) # "The Hateful Eight" with Samuel L. Jackson
172 ]

```

Figure 7 We finish our movie_genres list and create our movies_actors list.

```

173
174 # Insert data into tables using INSERT
175 cursor.executemany("INSERT INTO Movies VALUES (?, ?, ?, ?, ?)", movies)
176 cursor.executemany("INSERT INTO Genres VALUES (?, ?)", genres)
177 cursor.executemany("INSERT INTO Directors VALUES (?, ?, ?)", directors)
178 cursor.executemany("INSERT INTO Actors VALUES (?, ?, ?)", actors)
179 cursor.executemany("INSERT INTO MovieGenres VALUES (?, ?, ?)", movie_genres)
180 cursor.executemany("INSERT INTO MovieActors VALUES (?, ?, ?)", movie_actors)
181
182 # Commit changes and close the connection
183 connection.commit()
184 connection.close()
185
186 print("Data has been inserted successfully.")
187

```

Figure 8 We create our INSERT operations to then INSERT the data into the appropriate tables. Finally, we commit to and close the database. We have added a print function to provide the user with feedback that the operation is complete.

The following screenshots are of the populated data.

SQL ▾				
< 1 / 1 > 1 - 24 of 24				
MovieID	Title	ReleaseYear	Rating	
1	Jaws	1975	8.0	A giant great white shark terrorizes a small town.
2	E.T. the Extra-Terrestrial	1982	7.8	A troubled child summons the courage to help a friendly alien escape Earth and return to his home planet.
3	Goodfellas	1990	8.7	The story of Henry Hill and his life with the mob.
4	Taxi Driver	1976	8.3	A mentally unstable Vietnam War veteran works as a taxi driver in New York City while observing the actions of other people.
5	The Godfather	1972	9.2	The aging patriarch of an organized crime dynasty transfers control of his empire to his youngest son.
6	Apocalypse Now	1979	8.4	During the Vietnam War, Captain Willard is sent to locate a rogue American soldier who is rumored to have killed the South Vietnamese.
7	Pulp Fiction	1994	8.9	The lives of two mob hitmen, a boxer, a gangster's wife, and two lead patrons in a late 20th-century diner.
8	Kill Bill: Volume 1	2003	8.1	A former assassin awakens from a four-year coma and seeks revenge on her former employers.
9	Inception	2010	8.8	A thief who steals corporate secrets through the use of dream-sharing technology is given the inverse task of planting an idea into the mind of a CEO.
10	Dunkirk	2017	7.9	Allied soldiers are surrounded on the beaches of Dunkirk during World War II.
11	Gladiator	2000	8.5	A former Roman General sets out for revenge after being betrayed and sold into slavery.
12	Alien	1979	8.4	The crew of a commercial spaceship encounter a violent and deadly alien.
13	Fight Club	1999	8.8	An insomniac office worker and a part-time boxer in a seamy city forms a secret underground fight club.
14	Se7en	1995	8.6	Two detectives hunt a serial killer who kills people in the seven days of the week.
15	Spider-Man: No Way Home	2021	8.3	Peter Parker faces new challenges as he battles the Green Goblin and the other villains from his past.
16	Dune	2021	8.2	A noble family becomes embroiled in a war for planetary supremacy.
17	Unbreakable	2000	7.3	A man discovers he has superpowers.
18	Once Upon a Time in Hollywood	2019	7.6	An aging actor and his stunt double are caught in the changing tides of the Hollywood film industry.
19	Inglourious Basterds	2009	8.3	A group of Jewish soldiers in Nazi-occupied France during World War II.
20	Jurassic Park	1993	8.1	Scientists clone dinosaurs to create a theme park.
21	Blade Runner	1982	8.1	In a dystopian future, synthetic humans are indistinguishable from real humans.
22	Schindler's List	1993	9.0	In German-occupied Poland during World War II, a Jewish businessman saves the lives of his Jewish employees.
23	Reservoir Dogs	1992	8.3	After a botched robbery, tensions rise among the criminals.
24	The Hateful Eight	2015	7.8	In the aftermath of the American Civil War, a stagecoach driver is stranded in a remote frontier town.

Figure 9 This is our Movies table. We can verify that the number of entries from the script matches the entries of the table. The table was too wide to be captured in a single shot, but we can note the scroll bar at the bottom indicating that there is more to the table.

SQL ▼	
GenreID	GenreName
1	Drama
2	Crime
3	Thriller
4	Adventure
5	Sci-Fi
6	Action
7	Horror
8	War
9	Superhero
10	Epic
11	Comedy

Figure 10 This is our Genres table. The entries are consistent with our python script input.

SQL ▼		
< 1 / 1 > 1		
DirectorID	DirectorName	BirthDate
1	Steven Spielberg	1946-12-18
2	Martin Scorsese	1942-11-17
3	Francis Ford Coppola	1939-04-07
4	Quentin Tarantino	1963-03-27
5	Christopher Nolan	1970-07-30
6	Ridley Scott	1937-11-30
7	David Fincher	1962-08-28
8	Jon Watts	1981-10-28
9	Denis Villeneuve	1967-10-03
10	M. Night Shyamalan	1970-08-06

Figure 11 This is our Directors table. The entries are consistent with our python script input.

SQL ▼			< 1 / 1	
ActorID	ActorName	BirthDate		
1	Robert Shaw	1927-04-09		
2	Drew Barrymore	1975-02-22		
3	Ray Liotta	1954-12-18		
4	Robert De Niro	1943-08-17		
5	Marlon Brando	1924-04-03		
6	Martin Sheen	1940-04-03		
7	John Travolta	1954-02-18		
8	Uma Thurman	1970-04-29		
9	Leonardo DiCaprio	1974-11-11		
10	Fionn Whitehead	1997-01-18		
11	Russell Crowe	1974-04-07		
12	Sigourney Weaver	1949-10-08		
13	Edward Norton	1969-08-18		
14	Morgan Freeman	1937-06-01		
15	Brad Pitt	1963-12-18		
16	Samuel L. Jackson	1948-12-21		
17	Bruce Willis	1955-03-19		
18	Robert Duvall	1931-01-05		
19	Tom Holland	1996-06-01		
20	Timothée Chalamet	1995-12-27		
21	Harrison Ford	1942-07-13		

Figure 12 This is our Actors table. The entries are consistent with our python script input.

SQL ▼			< 1 / 1 > 1 - 49 of 49		
MovieGenreID	MovieID	GenreID			
1	1	7			
2	1	4			
3	2	4			
4	2	5			
5	3	2			
6	3	3			
7	4	3			
8	4	1			
9	5	2			
10	5	1			
11	6	3			
12	6	8			
13	7	2			
14	7	3			
15	7	12			
16	8	6			
17	8	3			
18	9	5			
19	9	3			
20	10	8			
21	10	3			
22	11	6			
23	11	1			
24	12	7			
25	12	5			
26	13	3			
27	13	1			
28	14	2			
29	14	3			

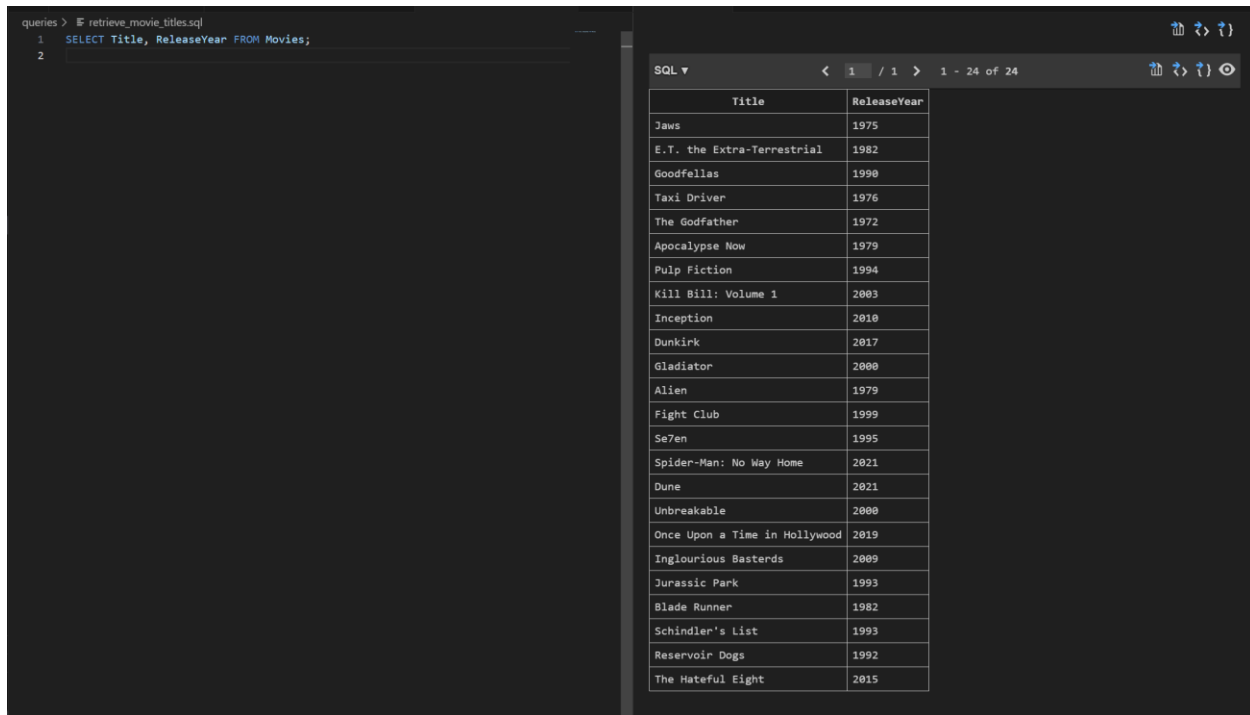
Figure 13 This is our MovieGenres junction table. We only captured the first 29 entries, but we see that they are consistent with our python script input. We can also note at the top a total of 49 entries, which is consistent with our input.

SQL ▼			< 1 / 1 > 1 - 33 of 33		
MovieActorID	MovieID	ActorID			
1	1	1			
2	2	2			
3	3	3			
4	3	4			
5	4	4			
6	5	5			
7	5	18			
8	6	5			
9	6	6			
10	6	18			
11	7	7			
12	7	8			
13	7	16			
14	7	17			
15	8	8			
16	9	9			
17	10	10			
18	11	11			
19	12	12			
20	13	13			
21	13	15			
22	14	15			
23	14	14			
24	15	19			
25	16	20			
26	17	17			
27	17	16			
28	18	9			
29	18	15			

Figure 14 This is our MovieActors junction table. We only captured the first 29 entries, but we see that they are consistent with our python script input. We can also note at the top a total of 33 entries, which is consistent with our input.

Section 4:

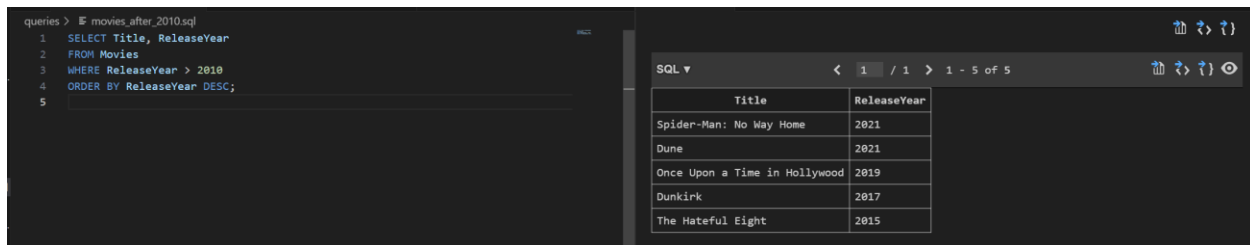
SQL Queries



The screenshot shows a SQL query editor on the left and a results pane on the right. The query is: `SELECT Title, ReleaseYear FROM Movies;`. The results pane shows a table with 24 rows and 2 columns: Title and ReleaseYear.

Title	ReleaseYear
Jaws	1975
E.T. the Extra-Terrestrial	1982
Goodfellas	1990
Taxi Driver	1976
The Godfather	1972
Apocalypse Now	1979
Pulp Fiction	1994
Kill Bill: Volume 1	2003
Inception	2010
Dunkirk	2017
Gladiator	2000
Alien	1979
Fight Club	1999
Se7en	1995
Spider-Man: No Way Home	2021
Dune	2021
Unbreakable	2000
Once Upon a Time in Hollywood	2019
Inglourious Basterds	2009
Jurassic Park	1993
Blade Runner	1982
Schindler's List	1993
Reservoir Dogs	1992
The Hatful Eight	2015

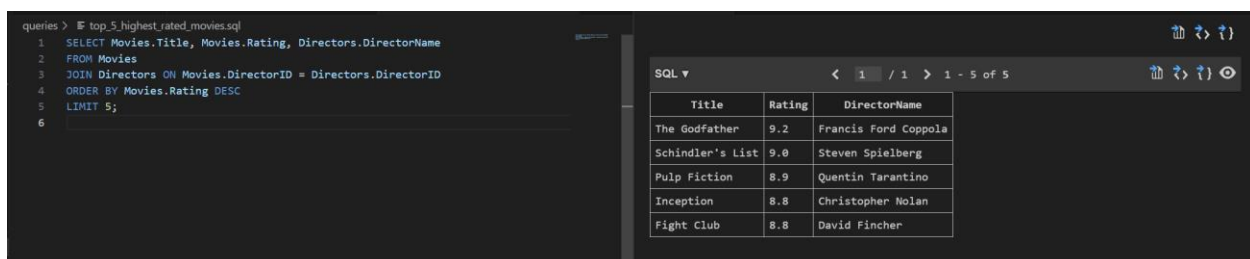
Figure 15 Retrieve all movies and their release dates.



The screenshot shows a SQL query editor on the left and a results pane on the right. The query is: `SELECT Title, ReleaseYear FROM Movies WHERE ReleaseYear > 2010 ORDER BY ReleaseYear DESC;`. The results pane shows a table with 5 rows and 2 columns: Title and ReleaseYear.

Title	ReleaseYear
Spider-Man: No Way Home	2021
Dune	2021
Once Upon a Time in Hollywood	2019
Dunkirk	2017
The Hatful Eight	2015

Figure 16 Movies released after 2010 in descending order.



The screenshot shows a SQL query editor on the left and a results pane on the right. The query is: `SELECT Movies.Title, Movies.Rating, Directors.DirectorName FROM Movies JOIN Directors ON Movies.DirectorID = Directors.DirectorID ORDER BY Movies.Rating DESC LIMIT 5;`. The results pane shows a table with 5 rows and 3 columns: Title, Rating, and DirectorName.

Title	Rating	DirectorName
The Godfather	9.2	Francis Ford Coppola
Schindler's List	9.0	Steven Spielberg
Pulp Fiction	8.9	Quentin Tarantino
Inception	8.8	Christopher Nolan
Fight Club	8.8	David Fincher

Figure 17 Top 5 highest-rated movies with their ratings and directors.

queries > <code>directors_with_at_least_3_movies.sql</code>	SQL 1 / 1 1 - 3 of 3								
<pre> 1 SELECT Directors.DirectorName, COUNT(Movies.MovieID) AS MovieCount 2 FROM Movies 3 JOIN Directors ON Movies.DirectorID = Directors.DirectorID 4 GROUP BY Directors.DirectorName 5 HAVING MovieCount >= 3; 6 </pre>	<table> <thead> <tr> <th>DirectorName</th><th>MovieCount</th></tr> </thead> <tbody> <tr> <td>Francis Ford Coppola</td><td>3</td></tr> <tr> <td>Quentin Tarantino</td><td>6</td></tr> <tr> <td>Steven Spielberg</td><td>4</td></tr> </tbody> </table>	DirectorName	MovieCount	Francis Ford Coppola	3	Quentin Tarantino	6	Steven Spielberg	4
DirectorName	MovieCount								
Francis Ford Coppola	3								
Quentin Tarantino	6								
Steven Spielberg	4								

Figure 18 Directors with at least 3 movies along with total movie count.

queries > <code>average_rating_per_genre.sql</code>	SQL 1 / 1 1 - 10 of 10																						
<pre> 1 SELECT Genres.GenreName, AVG(Movies.Rating) AS AverageRating 2 FROM Movies 3 JOIN MovieGenres ON Movies.MovieID = MovieGenres.MovieID 4 JOIN Genres ON MovieGenres.GenreID = Genres.GenreID 5 GROUP BY Genres.GenreName; 6 </pre>	<table> <thead> <tr> <th>GenreName</th><th>AverageRating</th></tr> </thead> <tbody> <tr> <td>Action</td><td>8.3</td></tr> <tr> <td>Adventure</td><td>8.0</td></tr> <tr> <td>Comedy</td><td>8.2</td></tr> <tr> <td>Crime</td><td>8.666666666666667</td></tr> <tr> <td>Drama</td><td>8.283333333333333</td></tr> <tr> <td>Epic</td><td>7.8</td></tr> <tr> <td>Horror</td><td>8.2</td></tr> <tr> <td>Sci-Fi</td><td>8.233333333333333</td></tr> <tr> <td>Thriller</td><td>8.433333333333333</td></tr> <tr> <td>War</td><td>8.15</td></tr> </tbody> </table>	GenreName	AverageRating	Action	8.3	Adventure	8.0	Comedy	8.2	Crime	8.666666666666667	Drama	8.283333333333333	Epic	7.8	Horror	8.2	Sci-Fi	8.233333333333333	Thriller	8.433333333333333	War	8.15
GenreName	AverageRating																						
Action	8.3																						
Adventure	8.0																						
Comedy	8.2																						
Crime	8.666666666666667																						
Drama	8.283333333333333																						
Epic	7.8																						
Horror	8.2																						
Sci-Fi	8.233333333333333																						
Thriller	8.433333333333333																						
War	8.15																						

Figure 19 Average rating per genre.

queries > F actors_in_genres.sql

```
1 SELECT DISTINCT Actors.ActorName
2 FROM Actors
3 JOIN MovieActors ON Actors.ActorID = MovieActors.ActorID
4 JOIN Movies ON MovieActors.MovieID = Movies.MovieID
5 JOIN MovieGenres ON Movies.MovieID = MovieGenres.MovieID
6 JOIN Genres ON MovieGenres.GenreID = Genres.GenreID
7 WHERE Genres.GenreName = 'Action';
8
```

SQL 1 / 1 1 - 3 of 3

ActorName
Uma Thurman
Russell Crowe
Tom Holland

Figure 20 Actors that have appeared in 'Action' movies.

queries > <code>movies_with_at_least_3_actors.sql</code>	SQL 1 / 1 1 - 2 of 2						
<pre> 1 SELECT Movies.Title, COUNT(MovieActors.ActorID) AS ActorCount 2 FROM Movies 3 JOIN MovieActors ON Movies.MovieID = MovieActors.MovieID 4 GROUP BY Movies.Title 5 HAVING ActorCount >= 3; 6 </pre>	<table> <thead> <tr> <th>Title</th><th>ActorCount</th></tr> </thead> <tbody> <tr> <td>Apocalypse Now</td><td>3</td></tr> <tr> <td>Pulp Fiction</td><td>4</td></tr> </tbody> </table>	Title	ActorCount	Apocalypse Now	3	Pulp Fiction	4
Title	ActorCount						
Apocalypse Now	3						
Pulp Fiction	4						

Figure 21 Movies with at least 3 actors from our list with the total number of actors.

Section 5:

Proposed Indices

Equality Query

Chosen Index: **Title** from table **Movies**

Rationale: Users frequently search for movies by their titles, making the Title column a common target for equality queries. Titles tend to be unique within a given dataset, which aids in optimizing searches. By indexing the Title column, the database can perform faster lookups, allowing it to quickly locate the row where the title matches the query.

SQL Statement: CREATE INDEX idx_movie_title ON Movies(Title);

Performance: It enhances speed by allowing the database to perform a binary search on a single column of a table, thereby reducing search time considerably. Additionally, it enables the database engine to swiftly locate the row(s) where the title matches the query, which in turn enhances performance for equality-based queries.

Range Query

Chosen Index: **ReleaseYear** from table **Movies**

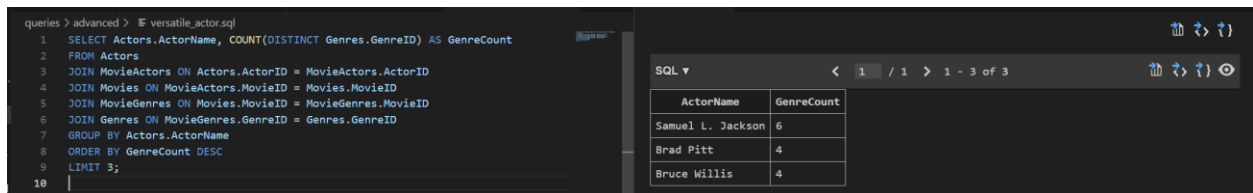
Rationale: The ReleaseYear column represents a continuous value, making it ideal for range queries, such as finding movies released within a specific time period. Users frequently search for movies within a certain release period, making ReleaseYear a common target for range queries. By indexing the ReleaseYear column, range-based searches become more efficient, as the database does not need to scan the entire table to retrieve the relevant data.

SQL Statement: CREATE INDEX idx_movie_release_year ON Movies(ReleaseYear);

Performance: The index allows the database engine to efficiently filter and retrieve rows that fall within the specified range, avoiding a full table scan. This greatly enhances the performance of queries that need to retrieve movies within specific time periods, providing a quick path to the relevant rows and ensuring faster query responses.

Section 6:

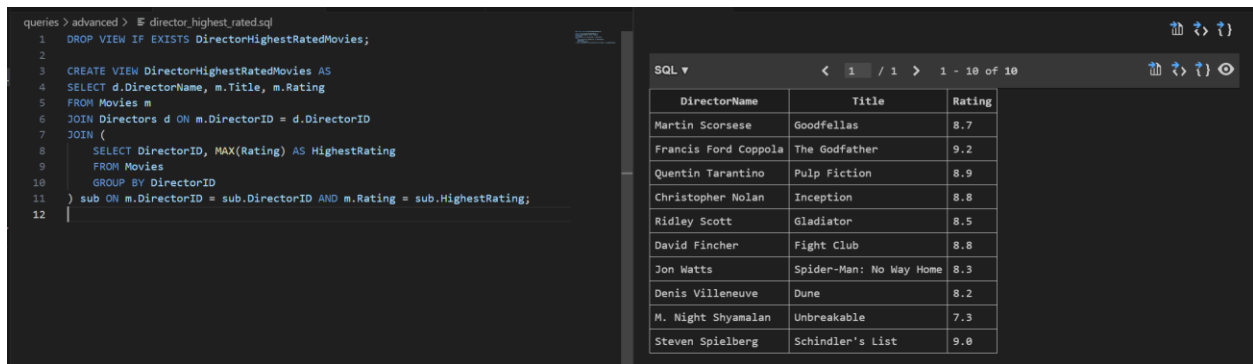
Advanced Queries



```
queries > advanced > versatile_actor.sql
1 SELECT Actors.ActorName, COUNT(DISTINCT Genres.GenreID) AS GenreCount
2 FROM Actors
3 JOIN MovieActors ON Actors.ActorID = MovieActors.ActorID
4 JOIN Movies ON MovieActors.MovieID = Movies.MovieID
5 JOIN MovieGenres ON Movies.MovieID = MovieGenres.MovieID
6 JOIN Genres ON MovieGenres.GenreID = Genres.GenreID
7 GROUP BY Actors.ActorName
8 ORDER BY GenreCount DESC
9 LIMIT 3;
```

ActorName	GenreCount
Samuel L. Jackson	6
Brad Pitt	4
Bruce Willis	4

Figure 22 Top 3 diverse actors (most genres).



```
queries > advanced > director_highest_rated.sql
1 DROP VIEW IF EXISTS DirectorHighestRatedMovies;
2
3 CREATE VIEW DirectorHighestRatedMovies AS
4 SELECT d.DirectorName, m.Title, m.Rating
5 FROM Movies m
6 JOIN Directors d ON m.DirectorID = d.DirectorID
7 JOIN (
8     SELECT DirectorID, MAX(Rating) AS HighestRating
9     FROM Movies
10    GROUP BY DirectorID
11 ) sub ON m.DirectorID = sub.DirectorID AND m.Rating = sub.HighestRating;
```

DirectorName	Title	Rating
Martin Scorsese	Goodfellas	8.7
Francis Ford Coppola	The Godfather	9.2
Quentin Tarantino	Pulp Fiction	8.9
Christopher Nolan	Inception	8.8
Ridley Scott	Gladiator	8.5
David Fincher	Fight Club	8.8
Jon Watts	Spider-Man: No Way Home	8.3
Denis Villeneuve	Dune	8.2
M. Night Shyamalan	Unbreakable	7.3
Steven Spielberg	Schindler's List	9.0

Figure 23 Create a view to see each director's highest rated movie with rating. Note we add a line to the beginning to ensure that we can generate the new view without conflict.

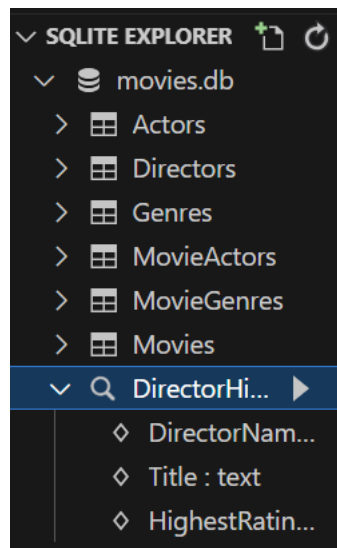


Figure 24 This is the view created in our database.

queries > advanced > incomplete_data.sql

1

2

3

4

5

6

7

8

9

```

SELECT
  Movies.Title,
  CASE WHEN MovieActors.ActorID IS NULL THEN 'Missing Actor' ELSE 'Has Actor'
  CASE WHEN MovieGenres.GenreID IS NULL THEN 'Missing Genre' ELSE 'Has Genre'
FROM Movies
LEFT JOIN MovieActors ON Movies.MovieID = MovieActors.MovieID
LEFT JOIN MovieGenres ON Movies.MovieID = MovieGenres.MovieID
WHERE MovieActors.ActorID IS NULL OR MovieGenres.GenreID IS NULL;

```

SQL

< 1 / 1 > 1 - 4 of 4

Title

ActorStatus

GenreStatus

Schindler's List

Missing Actor

Missing Genre

Reservoir Dogs

Missing Actor

Has Genre

Reservoir Dogs

Missing Actor

Has Genre

The Hateful Eight

Has Actor

Missing Genre

Figure 25 This query identifies whether a table does not have an associated actor or genre. We have used the following movies as test data to ensure that the query properly catches the cases we would like to identify.

Page 16 of 17

Section 7:

Challenges

Each of the advanced queries presented some different challenges that took some thought.

For the first advanced query, the main challenge was determining the distinct genres that each actor had appeared in. I ended up finding and using the COUNT(DISTINCT Genres.GenreID) function. This provided me with the ability to count the unique genres for each actor. This approach allowed me to focus on obtaining the correct count of distinct genres without needing to manually trace paths or connect tables.

For the second advanced query, I had trouble figuring out how to find the director's highest rated movie effectively. I did some research and decided to employ a subquery to help aggregate the information to contain the search for each author before generating the main table.

For the third advanced query, I wanted to identify if an item was missing an actor or genre to help narrow down the deficiency. After some online searching I decided to apply conditional statements. This provided me with what I needed to be able to isolate the deficiencies. I added additional movies to make sure the query worked properly.

Future Improvements

1) Awards / Nominations

- a. Award, Category, Nomination/Won, Award Year
- b. Adding awards and nominations will provide insights into the critical reception and recognition of movies, offering another dimension for evaluation beyond ratings.

2) Production Company / Studio

- a. Production Company: Name, FoundedYear, Headquarters, Notable Works
- b. Studio: Name, FK to Production
- c. Including details about production companies and studios will allow users to search for and evaluate movies based on the reputation and track record of the entities behind their creation. Examples: WarnerBros, 20th Century Fox, Disney

3) Financials

- a. Box-Office Gross, Budget, Profit
- b. Billion Dollar Movies: Benchmarks for success
- c. Budget and Profit: Key success metrics
- d. Range Queries: Enables detailed financial analysis
- e. Capturing financial metrics such as box-office gross, budget, and profit will enable a comprehensive analysis of a movie's commercial success and financial performance, facilitating range queries and trend analysis.